

PERIPHERAL EQUIPMENT

2023 – 2024

PRACTICE 1

By Adrián Sánchez López

1. Microcontroller's resources used

GPIOs

As GPIOs, the ports PG2, PG3, PG9, PD4, PD5, PA0 and PA4 have been used. PG2 and PG3 have been used as output ports through which PWM is displayed, port PG9 is the input auxiliary port used together with the button, which is PA0, ports PD4 and PD5 are the input ports that listen to PG2 and PG3 PWMs and finally PA4 is the analog output displaying V_c . PA0 has been selected because a button is needed and is the only usable button in the board, PA4 has been used as it is a DAC output port and the rest have been used as they are available GPIOs in the expansion connectors that do not have other features.

Timers

All general purpose timers have been used (TIM2 to TIM5), they have been used, as said, because they are general purpose ones and no special timer needed to be used. TIM2 and TIM5 are 32-bits long and TIM3 and TIM4 are 16-bits long. They can count up which is what I required. They have autoreload which is key for the TIM5 which has been used as the free-running timer that counts the time between rising edges from the PWMs and they can generate interrupts for different types of events, the only one I used is the overflow event, which is when the timer reaches the period assigned.

As said before, TIM5 has been used as the free-running timer, it means that it keeps counting up, which implies that it will eventually reach the maximum, when that happens extra operations need to be done to properly calculate the time since the last edge and the current edge, as the last would be a number close to 65535 and the current a value closer to 0. For example, imagine a last time value of 65035 and a current value of 300. If calculated as normal (current time - last time) we will get -64735, when the difference is actually 800. Therefore, before calculating the period it is checked if last time is bigger than current time (which only happens when the clock reaches its maximum and is reset). If it is, then the period is $(65535 - \text{last time} + \text{currentTime})$.

For the rest of the timers, TIM2 and TIM3 have been used to count up to T_p right and T_p left and throw an interrupt when overflowed, to generate the PWMs. Finally, TIM4 has been used to count up to 1 ms and throw an interrupt when overflowed, to toggle the status led.

DAC

The digital to analog converter has been used to transform the calculated value based on the current wheel's state to an analog output displayed through PA5, as said before.

EXTI

External interrupt/event controller has been to generate interrupts when the button is pressed or when the inputs listening to the PWMs detect a rising edge.

2. Interrupt structure

As previously said, EXTI interrupt and timer interrupts have been used. As there are different interrupts configured, there will be a lot of occasions where 2 interrupts are generated at the same time or 1 just after the other, which might make the one being executed second to not work as intended or measure wrong data as it is not being executed as soon as the interrupt is generated. To avoid these problems, priorities and sub-priorities have been arranged so in those cases the interrupt who depends more on being executed as soon as the interrupt is generated is executed first. The priorities have been the following, priority 0 has been given to the timers that generate the interrupts to generate the right and left sensor PWMs, the time for the left sensor has been given a subpriority of 0 and the one for the right of 1, so in case those 2 collide 1 is executed first. As the speed is generated in a way that makes the left wheel be faster or equal to the right wheel speed unless the button is pressed several times, a slightly superior priority is given to the left wheel timer as when testing it will be the fastest usually. Then, for PD4 and PD5 which generate interrupts when they detect a rising edge from the PWMs, they priority has been set to 1 and subpriority to 0 for the left and 1 for the right, again this subpriority follow the same logic as with the PWM timers. Finally, the timer that counts up to 1ms has been configured with a priority of 2 and sub priority of 0 and the interrupt generated when pressing the button with a priority of 3 and sub priority of 0.

The lower the number assigned the higher the priority, therefore the ones with the lowest priority are the timers used to generate the PWMs, followed by the interrupts generated at each PWM rising edge to count its period, the 1 ms timer and finally the button interrupt. This has been done like that as delaying the interrupt used to generate the PWM would result in a wrong PWM, then the second most important interrupt is the one used to count the period of each PWM, as if delayed it will result in a longer or shorter period. Note that both the interrupt to generate the PWM and count its period require 0 delay, but if the interrupt to generate the PWM is delayed, not only the PWM will be generated wrong, but also the period counted for that PWM will be wrong, as it is being counted from a wrong PWM. Therefore, interrupt to generate the PWM has higher priority than the one to count its period, as the first affects the second, but not the other way around. Then we have the interrupt of the 1ms timer and the interrupt for the button. The one to count up to 1 ms comes first as if delayed it will not count up to 1ms, but the PWM and its period are more important to be accurate than the 1ms counter as the PWMs are key for the development of the project, but the 1ms counter is only used to toggle a led each 200 ms, therefore if it gets delayed 1,2 or 3 times each 200 ms it won't matter that much, as first its function is less important and it counts up to 200 ms, so the delay generated for executing another ISR first won't affect that much this functionality as in the PWMs PWM has smaller times. Finally, the button has the worst priority as it is just required that when pressed the speed or correlation between wheel speeds is updated, as far as this is achieved, even if it is done with a small delay it will not affect the functioning of the system and we humans won't be able to perceive it.

: the interrupts used, relative priorities, etc.

3. Answers

Execution time of each ISR.

To get the execution time of each ISR, a GPIO has been set in the first line of each ISR and reset at the end. Then, the execution time is taken by measuring the time from the rising edge until the falling edge of that GPIO.

Execution time TIM2_IRQHandler(): 1 us

Execution time TIM3_IRQHandler(): 1us

Execution time TIM4_IRQHandler(): 1,12 us

Execution time EXTI0_IRQHandler(): 6,96 us

Execution time EXTI4_IRQHandler(): 166 ns

Execution time EXTI2_IRQHandler(): 166 ns

Rotation speed sensor ISR latency.

To be able to assess the latency of each wheel rotation speed sensor, a GPIO was set when the interrupt that generates the pwm sets the GPIO of the pwm and then in the interrupt of the rotation speed the GPIO set is reset.

The latency for the left wheel sensor has been 687 ns, while the latency for the right wheel has been 837 ns. There is a difference between the left and the right, which is about 150 ns which is the execution time of EXTI4. Which makes sense as the right wheel has the latency of the left one (as the process producing the latency is the same for both) + the execution time of the left one, as the left one has more priority so the right one has to wait for the left one to finish.

CPU Bandwidth dedicated to interrupt management.

$$\text{Bandwidth} = \text{Lavg} * (\text{toh} + \text{tisir})$$

The bandwidth may change depending on the user actions, therefore min and max bandwidth will be calculated. Min bandwidth is the bandwidth of TIM4 which interrupt is executed each 1ms. EXTI0 (button int) is not counted as the minimum is when the user does not press the button. Also EXTI4, EXTI2 are not executed as the pwm is not generated as speed is 0. TIM2 & 3 are executed but no pwm is generated.

$$\text{MiniBandwidth} = \text{bandwidthTIM4} + 2 * \text{bandwidthTIM2}$$

$$\text{BandwidthTIM4} = 1000 * (0 + 1,12 * 10^{-6}) = 1,12 * 10^{-3}$$

$$\text{BandwidthTIM2} = 1000 * (0 + 1 * 10^{-6}) = 1 * 10^{-3}$$

Lavg is 1000 as TIM4 generates interrupt each ms and TIM2 generates interrupt each ms when speed is 0. TIM3 is not calculated as it is the same as TIM2, so instead TIM 2 is multiplied by 2

$$\text{MinBandwidth} = 2 * 10^{-3} + 1,12 * 10^{-3} = 0,00312.$$

Max bandwidth is bandwidth of TIM4 interrupt, plus TIM2 and TIM3 interrupts when speed is 270 and correlation 2,2 and BanEXTI0 + ban_EXTI4 and ban EXTI3

MaxBandwidth = Ban_TIM4 + Ban_TIM2 + Ban_TIM3 + Ban_EXTI0 + Ban_EXTI4 + Ban_EXTI3.

BanTIM4 = 0,001.

BanTIM2 = $5400 * 1 * 10^{-6} = 0,0054$. 5400 is the the number of interrupts generated with speed 594 km/h ($270 * 2,2$) each second.

BanTIM3 = $2450 * 1 * 10^{-6} = 0,00245$. 2450 is the number of interrupts generated with speed 270 each second.

BanEXTI0 = $1 * 6,96 * 10^{-6} = 0,00000696$. Assume in the worst case the user will press the button 1 time per second.

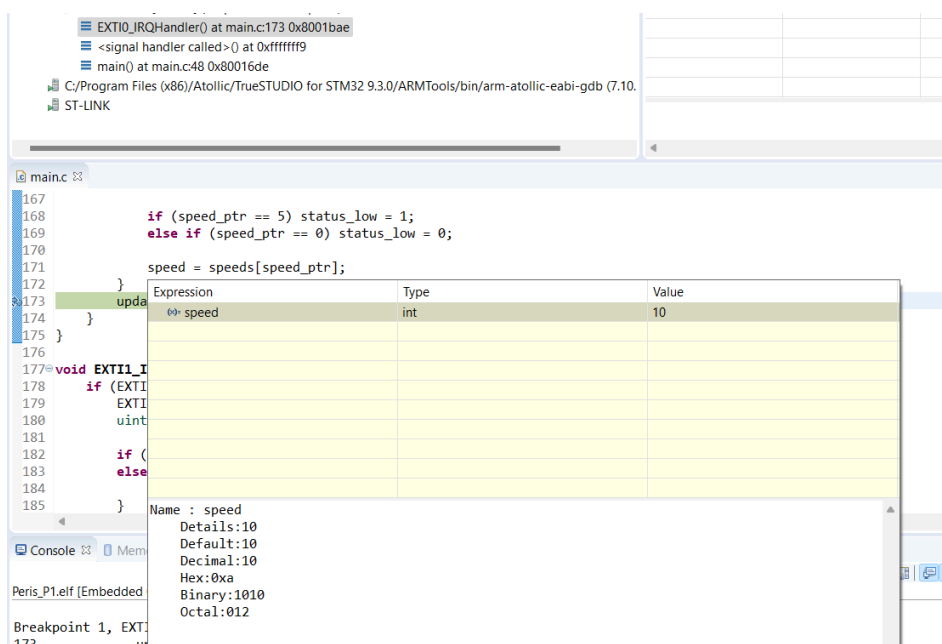
Ban_EXTI4 = $2700 * 1,66 * 10^{-7} = 0,00044$

Ban_EXTI3 = $1225 * 1,66 * 10^{-7} = 0,000203$

MaxBandwidth = $0,001 + 0,0054 + 0,00245 + 0,00000696 + 0,00044 + 0,000203 = 0,0095$.

Mechanism to visualize the contents of the variables.

When debugging and when in a breakpoint you can put the cursor over the variable/ expression you want and its result will be shown. As seen on the picture, when leaving the cursor over a variable/expression when in a breakpoint a new window appears with the variable/expression, its type and its value, plus then below the value in different numeric systems(binary, decimal, hexadecimal, octal).



Alternatively, as explained in the Atollic guide, printf can be used with the proper set up explained in the guide to print variable values.

4. Mechanisms to modify the variables

Variables are modified as usual, no strange mechanism is needed, when variables need to be accessed from an ISR, the variables are declared globally, so the ISR that needs them can access them. Also when wanting to get 2 values as with the speed of right and left wheel, a function is called passing 2 variables(speed_right, speed_left) as reference, so with 1 call to the function it can get both variables without having to return an array or having to call the function 2 time(1 per each speed).

PERIPHERAL EQUIPMENT

2023 – 2024

PRACTICE 2

By Adrián Sánchez López

1. Practice 1 changes

Regarding its behavior, some changes were made regarding the practice 1 generation of the PWM. As before, I had an array storing the different speeds in Km/h from where the speed of the car was taken depending on the number of times the button was pressed. Therefore, in order to generate the PWM's this speed had to be converted from Km/h to m/s to revolutions to period and then halved. As these operations take an incredibly high amount of time, to further optimize the project I changed the speeds stored in the array to the periods halved, so now there is nothing to be calculated, the value of the speed is the value I directly use to generate the PWM.

When it comes to GPIOs, the ports PD4 and PD5, which are the input ports that listen to left and right wheel PWMs have been changed to the ports PC0 & PC1 as while taking out the operations done to generate the PWM(as explained before and which do not affect in anything PD4 and PD5 interrupt), one stopped working while the other was generating an interrupt constantly. Due to that, I ended up changing those ports with the ports PC0 and PC1.

2. ADC Implementation

For the ADC the ADC1 has been used, I could have used any other one, but this was just the first to appear in the channels of stream 0 of the DMA which is the DMA used for peripheral to memory transfer. Also, to take a sample of each sensor each 50 us during 8 times each 1ms, the timer of 1 ms has been used to set a flag and a timer that throws an interrupt each 50 us has been implemented. Then in the interruption of this timer of 50 us, if the flag of the timer of 1ms is set and less than 8 samples have been taken it starts an ADC conversion. If 8 samples have been taken and the flag of the 1ms timer is set, it resets the number of samples taken to 0, unsets the flag of the 1ms timer and enables the DMA2_stream1 which is the one that makes the memory transfer. Note that the adc is configured to make the conversions on multiple channels, and to not start a conversion after one, so the conversion is done just once and when I call the function to start the conversion in the interrupt of 50us under the appropriate conditions.

For what it has to do with the ports, the ports PA1 and PA2 have been used as those are the input1 and input2 ports for each of the ADC. They have been set as analog and in the adc the channels 1 and 2 which, as said, are inputs PA1 and PA2 have been set.

3. DMA Implementation

From all the DMA's, the DMA2 has been used as it is stated that it is the only DMA that allows for memory to memory transfers. Then the stream0 channel0 is used to transfer from peripheral to memory. Stream0 channel0 is explicitly used, as it is the channel connected to ADC1. Then the stream1 channel1 is used to transfer from memory to memory, as it is memory to memory, no peripheral is used, so no specific channel attached to a peripheral is needed.

Regarding DMA stream0 channel 0, it has been set as peripheral to memory as said, with the proper addresses of both the peripheral and the memory where I want the samples stored. The buffer size is set to 16 bits as it is the closest option towards the 12 bits that each sample occupies. The memory increment is enabled as I want the dma to increase the address of the array for each transfer and the peripheral inc is disabled as the address of the ADC will remain always the same. Finally, it has been set as circular mode as I want the DMA to reset the data counter to 0 when it reaches the end of the array where it stores the samples from the adc.

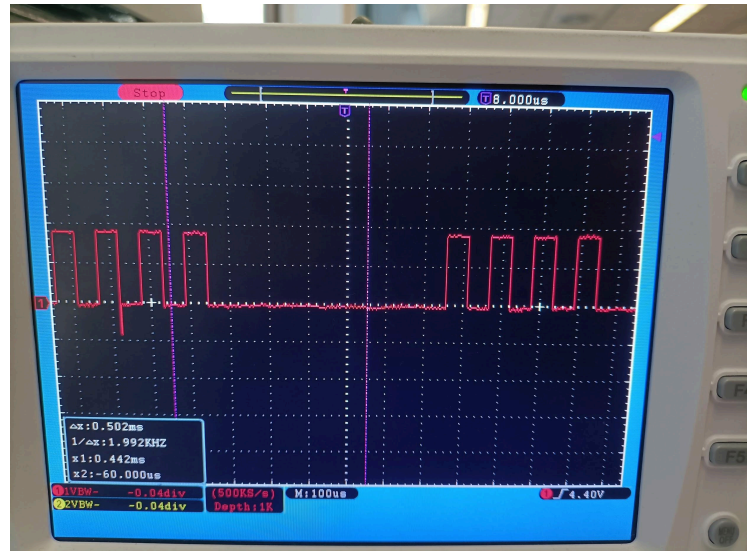
Regarding DMA stream1 channel1, it has been set as memory to memory, with the proper address of the array where the samples are and the array where I want it to transfer and store them. For both arrays I have the memory increment enabled as I want it to increment the position and it is required for the burst mode. The data sizes have been set to 16 bits and the mode is normal, so it makes the transfer each time I enable it, and once done it is disabled. And the burst is set to single so each I enable it transfers the whole array at once and gets disabled until I want to enable it again for the next transfer.

In addition, for the stream1 flag that indicates the completion of a transfer, I created an interrupt so each time I enable the stream1 for it to transfer the whole data in the adc timer when 8 samples have been taken, after they are transferred it throws an interrupt in which I calculate the mean value of the 8 samples.

4. Validation methods

To check that the ADC converted the values well, with the debugger I checked that the value generated after the conversion was consistent with the voltage being applied to the port. In addition, to check that the ADC acquires the samples at the specified intervals and at the requested rate, I toggled the free led available each time a conversion is done to check with the oscilloscope that each millisecond there where 8 consecutives toggles in the signal, each one done each 50us.

As it can be seen in the picture, there are 4 rising and falling edges, so in total 8, which is the number of samples(remember that each toggle to the signal is made after a sample is done). Moreover, each change happens half a square after the previous change and each square is 100us. Therefore, each change occurs after 50us. Finally, after the 8th change(samples) the next change occurs just 6 squares(600 us) later, which adds to the 400 us that have already passed while taking samples, it adds up to 1ms which is the rate at which samples must be taken.



In addition, it also has been put a bit toggle on the free led when the stream1 transfer(memory to memory) is made and as seen in the result, each transfer is made 1 ms after the previous.



Finally to check that both arrays had the same data after each transfer the debugger was used to check both array values. And to check that in the first array the values of the conversions where being stored in the correct order the debugger has been used too, to check that the values follow the order as specified in the table.

samples	uint16_t [16]	0x2000004c <samples>
samples[0]	uint16_t	3951
samples[1]	uint16_t	7
samples[2]	uint16_t	3949
samples[3]	uint16_t	7
samples[4]	uint16_t	3
samples[5]	uint16_t	3
samples[6]	uint16_t	4
samples[7]	uint16_t	4
samples[8]	uint16_t	5
samples[9]	uint16_t	5
samples[10]	uint16_t	6
samples[11]	uint16_t	6
samples[12]	uint16_t	7
samples[13]	uint16_t	7
samples[14]	uint16_t	8
samples[15]	uint16_t	8

Firstly, in this picture it can be seen how the values of the conversion are stored correctly from two to two and in the correct order.

samples	uint16_t [16]	0x2000004c <samples>
samples[0]	uint16_t	3951
samples[1]	uint16_t	7
samples[2]	uint16_t	3949
samples[3]	uint16_t	7
samples[4]	uint16_t	3959
samples[5]	uint16_t	6
samples[6]	uint16_t	3957
samples[7]	uint16_t	5
samples[8]	uint16_t	3958
samples[9]	uint16_t	7
samples[10]	uint16_t	3953
samples[11]	uint16_t	6
samples[12]	uint16_t	3955
samples[13]	uint16_t	7
samples[14]	uint16_t	3959
samples[15]	uint16_t	6

samplesCopy	Sample [8]	0x2000012
samplesCopy[0]	Sample	{...}
sensorX	uint16_t	3951
sensorY	uint16_t	7
samplesCopy[1]	Sample	{...}
sensorX	uint16_t	3949
sensorY	uint16_t	7
samplesCopy[2]	Sample	{...}
sensorX	uint16_t	3959
sensorY	uint16_t	6
samplesCopy[3]	Sample	{...}
sensorX	uint16_t	3957
sensorY	uint16_t	5
samplesCopy[4]	Sample	{...}
sensorX	uint16_t	3958
sensorY	uint16_t	7
samplesCopy[5]	Sample	{...}
sensorX	uint16_t	3953
sensorY	uint16_t	6
samplesCopy[6]	Sample	{...}
sensorX	uint16_t	3955
sensorY	uint16_t	7
samplesCopy[7]	Sample	{...}
sensorX	uint16_t	3959
sensorY	uint16_t	6

Then by comparing those two images showing the first the contents of the first array and the second the contents of the array where they are transferred, it can be seen how both have the same values.

PERIPHERAL EQUIPMENT

2023 – 2024

PRACTICE 3

By Adrián Sánchez López

1. Program architecture and data structures

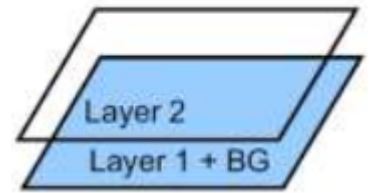
For the data acquisition in phase2 I already created an interrupt for the DMA stream that transferred the samples from memory to memory, and there I got the mean of the 8 samples. Therefore, I added 2 global variables, numSamples and numTransfers and in the interrupt of the DMA, numtransfers is incremented by 1 each time I get the 8 samples and calculate the mean. So when numTransfers is 10, I increment numSamples, set numTransfers to 0 and call the function displaySamples that updates the waveforms of both accelerations. In addition, when numSamples is 200 a flag that tells the functions to display the accelerations that we already received 200 samples is set. Finally, each time that the number of samples modulus 10 is equal to 0, the function that displays the bitmap in the x-y graph is called.

Regarding the display, the new average of the sample is passed as a parameter to the function displaySamples. In addition there is a global array that stores 201 Positions, which is a structure that stores the position corresponding to sampleAverage passed for both accelerations. DisplaySamples firstly checks if the flag I previously mentioned that tells the program if at least 200 samples have been received. If it has received less, then it calculates the position of the new sample for both graphics, stores it in the next free position of the array storing them and displays the pixel in the calculated position for both waveforms. On the other hand, if 200 samples have been received, then the new sample positions are calculated and stored in the array in the last position(position 201), and then a for loop goes through the whole array shifting the positions of the array to the left. While doing so it unsets the previous pixel position and sets them in their new position.

Regarding the x-y graph, when in the previous function the positions were calculated for both waveforms depending on the sample received, it also stored that sample in an array of 10 samples, which works like a LIFO. As mentioned before, when numSamples modulus 10 is equal to 0(100 ms have passed), the function that displays the bitmap is called, this functions calculates the average value of the 10 samples stored in that array that works like a LIFO and calculates the position of the value. Once calculated, it calls the function DrawBitmap to clear the previous bitmap and calls the DrawBitmap again to paint the new Bitmap. Finally, the position painted is stored in a variable that stores the previous bitmap position, so the next time this function is called, the bitmap painted now can be cleared.

2. Video subsystem configuration

As seen in the following diagram, 2 layers have been used, the first being the background that has rectangles that enclose both waveforms and the x-y graph painted in black and the rest painted in white. The second being a transparent layer where the waveforms and the bitmaps are painted and clear and updated, so no need for repainting the whole layer is needed, only the painted pixels to be updated are cleared and the painted in their new position. The first layer which is the background uses the address 0xD0000000 and the second one uses the address 0xD0050000. As said just now, the second layer is transparent except for the pixels needed for the waveforms and bitmap, that way the background doesn't need to be repainted at all and the few painted pixels from layer2 are the ones that are individually cleared and repainted. To do so, the alpha value has been used at the start to set all the pixels of the layer as transparent and to set as transparent the pixels painted that need to be cleared.



3. Implementation

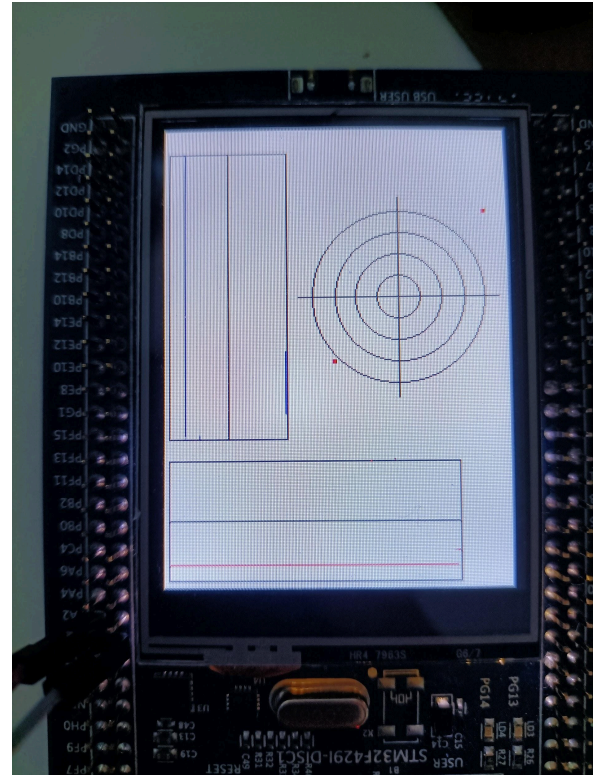
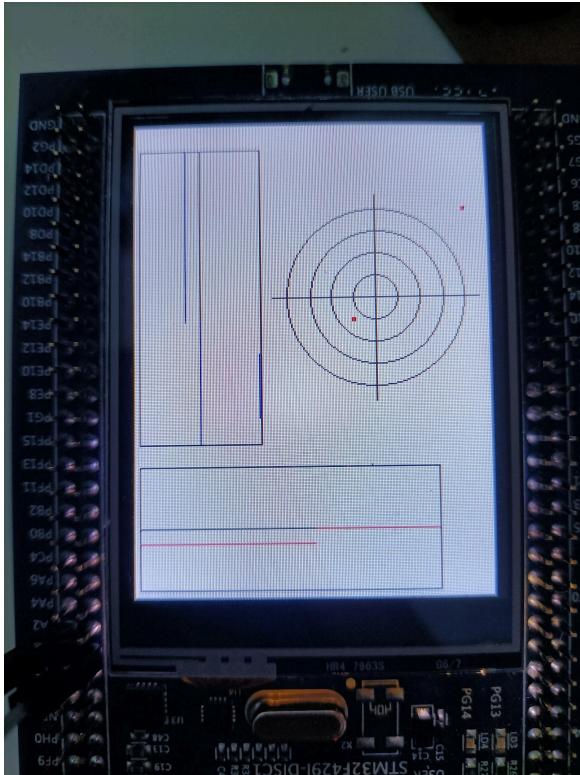
The requested pixel format is ARGB4444, which means that it uses 4 bits for Alpha value, 4 for the red value, 4 for the green value and 4 for the blue value. Therefore in total each pixel is made of 16 bits(2 bytes), so if I want to access the pixel in the row 5 and column 6, then I calculate the number of pixels(6 rows * pixelsPerRow + 6 columns(6 pixels)) and then the number of pixels is multiplied by the size of each pixel(2 bytes). So if the width is 240 pixels, then $6 * 240 + 6 = 1446$ pixels * 2 bytes per pixel = 2892 bytes, this will be the offset that needs to be added to the address of the layer I want to draw on. Another interesting point is how to set up the 2 bytes conforming the pixel, as the functions received ARGB in a format of 8 bits and I want them in a format of 4 bits, I shift each of them 4 bits to the right so I only keep the 4 most significant bits. Then, with logical ORs I add the alpha value shifted to the left 12 bits, the red value shifted to the left 8 bits, the green value shifted to the left 4 bits and the blue value without shifting, with that I conform the pixel in the ARGB4444 format.

```
((alpha >> 4) << 12) | ((red >> 4) << 8) | ((green >> 4) << 4) | (blue >> 4);
```

I previously explained how to access a pixel to modify it, but that was to modify the whole pixel, if you only want to make the pixel transparent for instance. Then you only require the alpha value, as it is going to be transparent the RGB values will not be used for anything. In this case you will have to calculate the address of the desired pixel as said before, but as I only want to modify the alpha I will only set the 4 bits of the alpha value. If for instance I wanted to change only the green value, then as it is located after the 8 MSB, then I will calculate the address as explained, but I will add an extra 8 bits which are the 8 bits of the alpha + the red which are located before the green, and then set the 4 bits of the green.

Finally, for drawing the horizontal or vertical lines, the procedure is as follows: it will first check if the starting pixel and the ending pixel are inside the screen. If they aren't, then it returns a NO_OK. If they are, then it checks if the row/col_start is bigger than the row/col_end, if the start is bigger then it switches both. Afterwards, a for loop calls the function setPixel to set the pixels from the start point until the endpoint to the desired ARGB. If I wanted to be more efficient, then instead of calling the function set pixel that will calculate the address each time, then I could calculate the offset for the starting pixel and then multiply it by 2 for each of the following pixels until the end point. That way it would waste less time calculating the address.

4. LCD



5. Conclusions

Overall, this practice has helped me deepen my knowledge in data visualization devices and the architecture of their controllers. Developing a simple graphic library has made me understand even more how pixels work, ways to optimize different procedures such as accessing a pixel, drawing a line, drawing a circle... Not only that, but this has also been the first time I implemented a graphic library, though basic, it has laid the foundations towards making more complex graphic procedures in the future.